



Which car should you get?

Dispatch rules for destination-controlled elevators, measured

elevator-dispatch-sim · one engineer, one AI assistant, one brief

THE QUESTION

How should a bank of elevators decide which car takes which passenger — and how much does the choice matter?

THE ANSWER, MEASURED

1.15–3.6x

shorter average wait with **smart dispatch**
than **nearest car**

2–3.3x

shorter than **take turns** — on every realistic
traffic pattern tested

The problem in plain language

Destination dispatch

You enter your destination before you board. The controller picks one car, at that moment, and the choice is final.

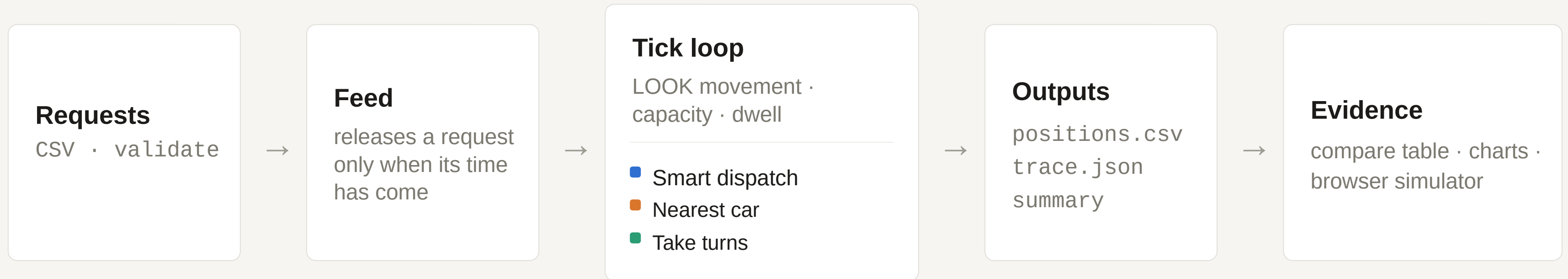
What good means

Short **wait** — request to boarding.
Short **total time** — request to arrival.
And **fairness** : no one waits far longer than everyone else.

Why it is not obvious

The controller cannot see the future. And the goals pull apart: the best average bundles people onto a busy car, which is efficient on the whole and hard on whoever keeps being passed over.

What was built



3

rules behind one interface

10

seeded scenarios, byte-reproducible

131

tests, six levels

0

runtime dependencies

How the model works

for the engineers • skippable

ONE TICK, IN FIXED ORDER

- 1 **Release and assign** — requests due now reach the scheduler; it returns a car index.
- 2 **Unload, then load** — at the current floor, in request order, while capacity allows, only in the departure direction.
- 3 **Log** — one row of positions per tick, from tick 0.
- 4 **Advance** — each car moves one floor toward its next stop, dwells, or stays idle.

ENFORCED BY STRUCTURE, PINNED BY TESTS

No peek-ahead. The feed releases a request only when $\text{time} \leq \text{now}$; a spy scheduler proves it.

Assignment is final, as in real destination systems — so capacity is the scheduler's problem.

LOOK movement. Keep direction while a stop lies ahead; never carry anyone away from their destination.

A stop costs one tick. The brief's literal model is recovered with `--dwell 0`.

The three dispatch rules

Smart dispatch

ETD · estimated time to destination

Picks the car that delays everyone the least: the newcomer's projected time to destination plus the delay added to passengers already assigned.

Trade-off: optimises the aggregate; without a fairness weight it can leave one person waiting a long time.

Nearest car

baseline · distance

Sends the closest car going the right way; a large penalty if the car is moving away or in the wrong direction.

Trade-off: ignores how many stops the car already has — so it degrades as load rises.

Take turns

baseline · round robin

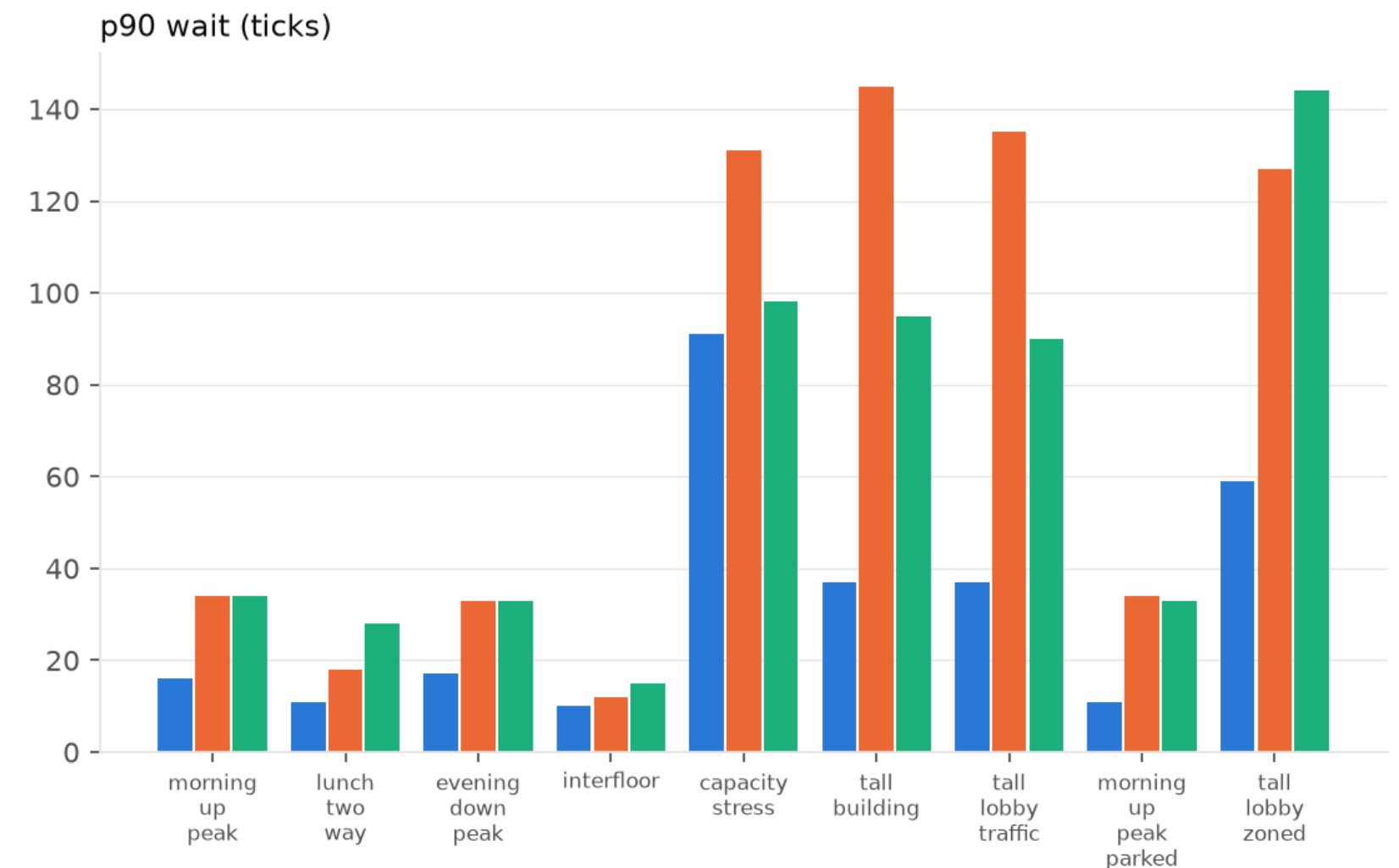
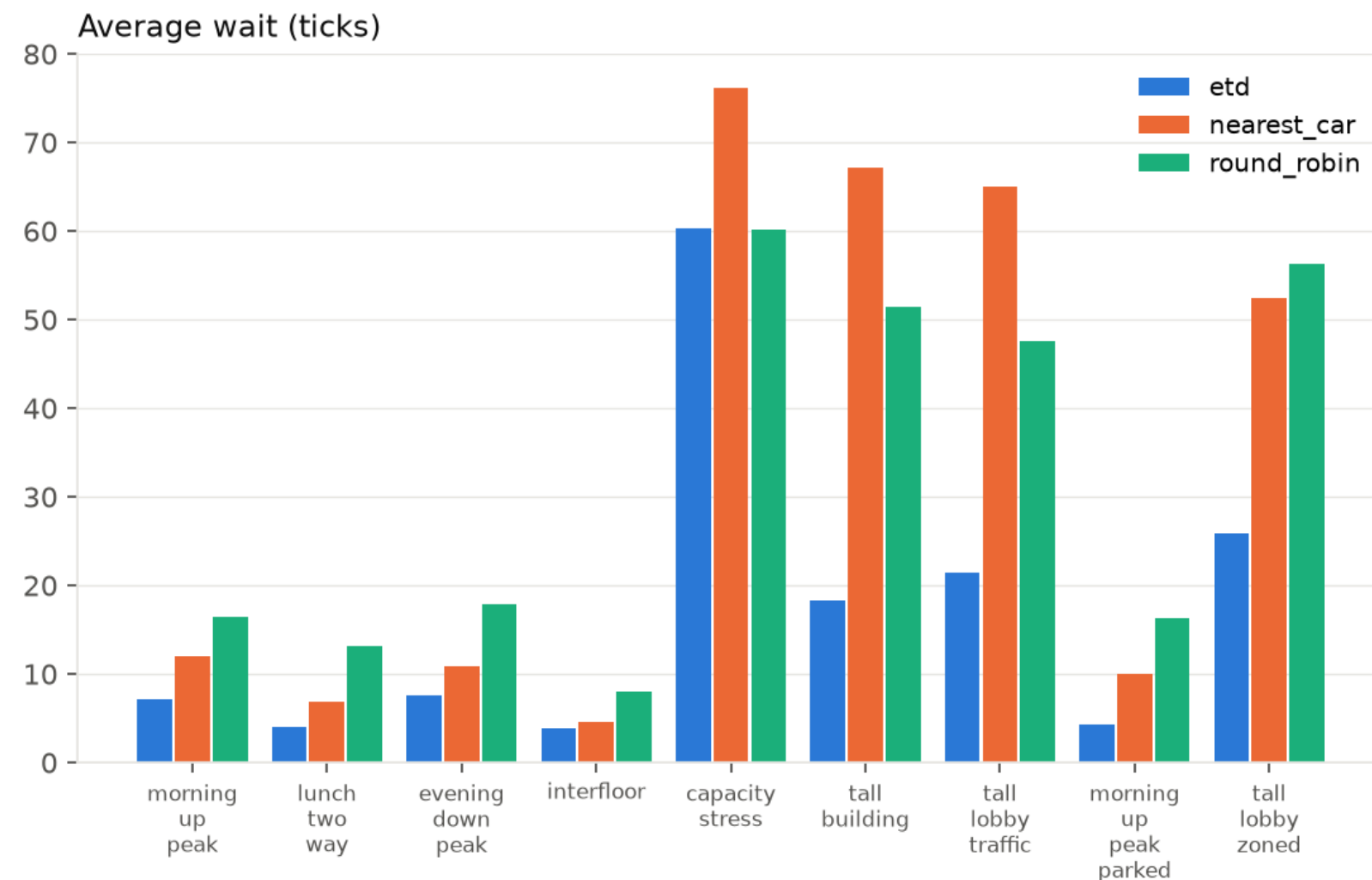
Cars take requests in strict rotation, wherever they are. It uses no information at all.

Trade-off: nobody is singled out for a long wait; everybody waits longer.

Three policies sit on top of any rule: a **fairness weight** that makes long waits progressively more expensive to extend, **parking** idle cars at a chosen floor, and **zoning** cars to a range of floors.

Waiting time by scenario

ticks · identical traffic per rule



■ Smart dispatch ■ Nearest car ■ Take turns

Lowest on every realistic pattern; the margin grows with height and load.

Average wait per rule, per pattern

ticks · lowest in bold

Scenario	Smart	Nearest	Turns
Morning up-peak · 20 floors, 4 cars	7.1	12.0	16.5
Lunch two-way · 20 floors, 4 cars	4.0	6.9	13.2
Evening down-peak · 20 floors, 4 cars	7.6	10.8	17.9
Quiet interfloor · 20 floors, 4 cars	3.9	4.5	8.1
Tower · 51 floors, 6 cars	18.3	67.1	51.4
Lobby burst · 20 floors, 2 cars × 6	60.3	76.2	60.1

TWO CASES THAT GO THE OTHER WAY

Lobby burst: a tie

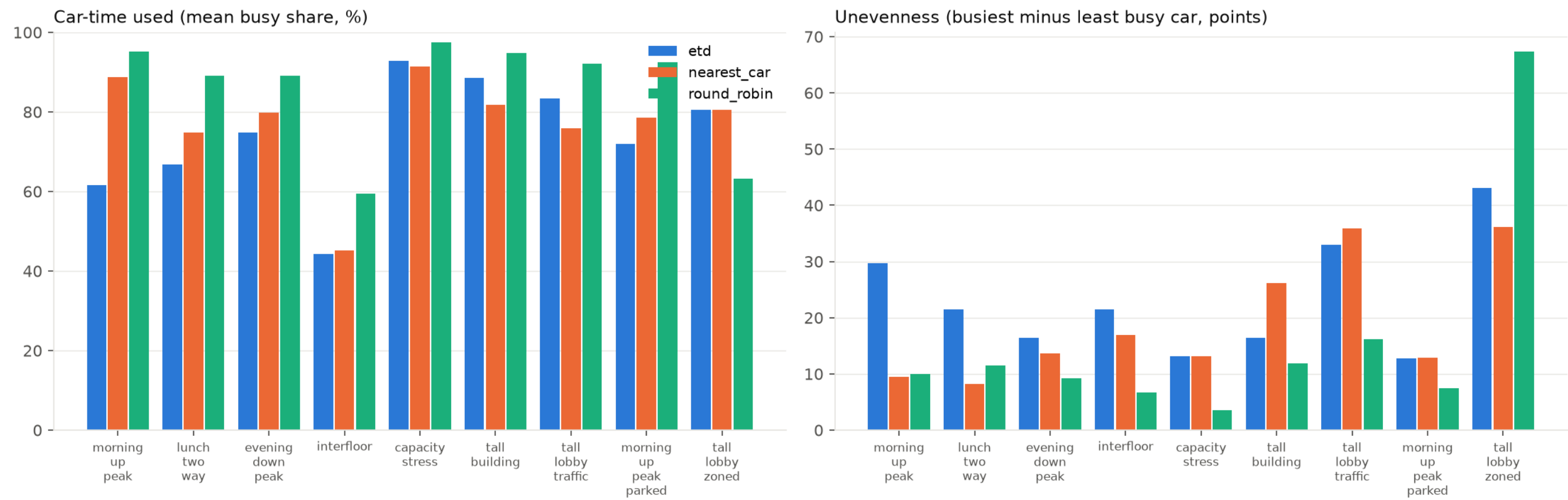
60.3 against 60.1. Every request has the same origin and the system is saturated; the only lever left is spreading load evenly, which take turns does by construction.

The brief’s sample: nearest car wins

6.3 against 15.0 on three passengers. Smart dispatch splits two lobby passengers across both cars to save a stop, leaving no idle car for the third request. A greedy rule with no view of the future can lose on tiny inputs — which is why the scenarios, not the sample, are the basis for judgement.

How the cars are used

busy = aboard or assigned · spread = busiest – least busy car



62% vs 89% vs 95%

Least car-time for the same job. Cars busy in the morning peak; fewer stops per trip is less travel per passenger.

36% fair share 25%

But the work is shared least evenly. One car in four carries 36% of the morning passengers.

30 → 18 · 21 → 5 · 33 → 10

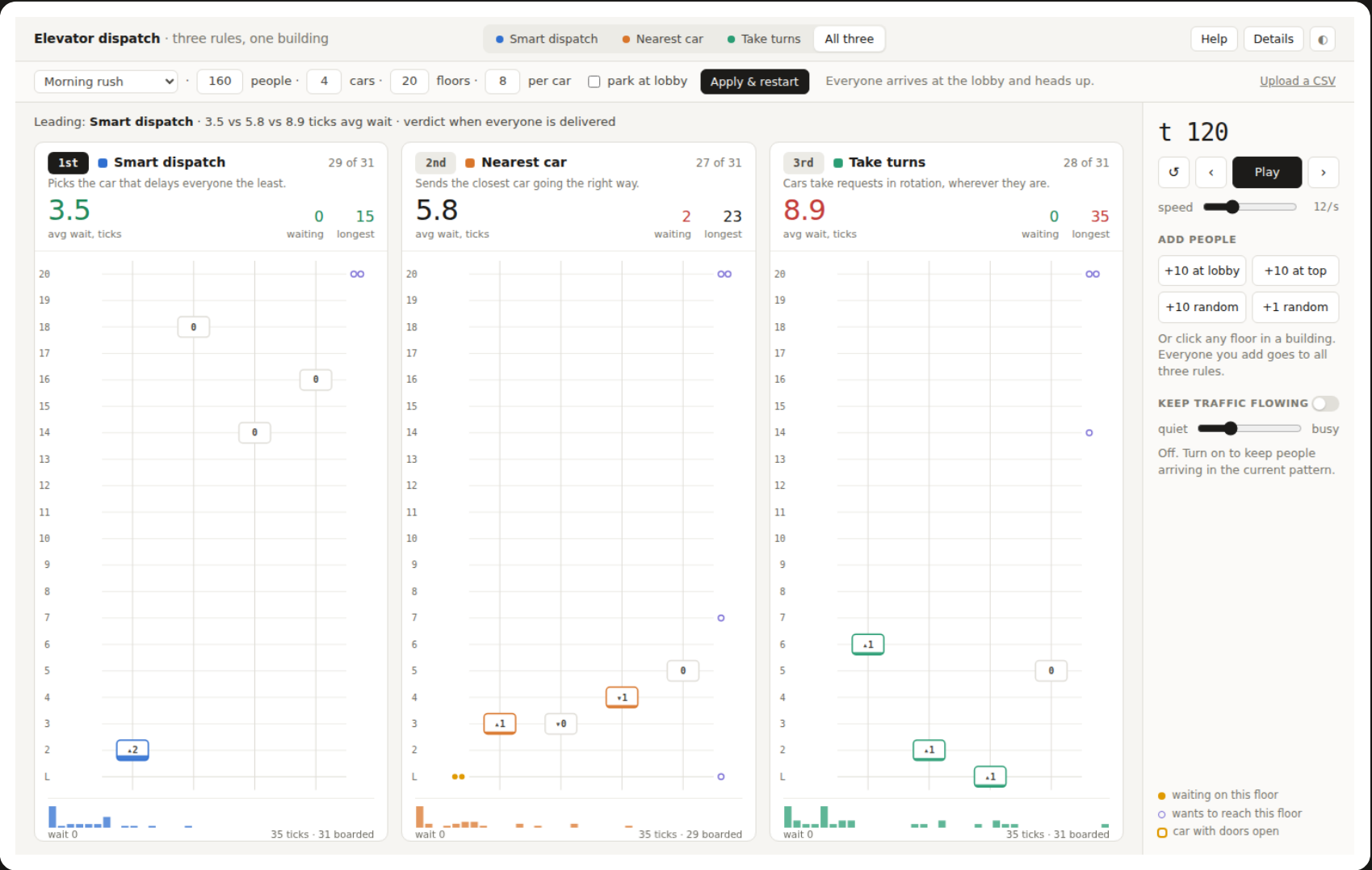
The fairness weight evens the cars as a side effect. Spread in points — morning, lunch, tall lobby. One knob, two benefits.

The live simulator

`samuelgregorovic.github.io/elevator-dispatch-sim/docs/viewer/`

WHAT TO WATCH FOR

- Three rules, one building, the same 160 people — the **ranking** updates live.
- Amber dots** are people waiting. Watch where they pile up under each rule.
- The **usage strip** under each building: one bar per car, filled by busy share — see which cars sit out.
- Click a floor to **add a passenger**; tick **park at lobby** and restart.



Which rule, which policy, for which building average wait, ticks

Building and traffic	Choose	Evidence
Ordinary office, moderate traffic · ≤20 floors	Smart dispatch. 29–42% shorter average wait than nearest car on the three peaks; p90 halved in morning and evening.	Strong · four patterns, one direction
Quiet, interfloor traffic	Smart dispatch , but by 13% — a nearest-car controller is within 15% here.	One pattern
Tall building · ~50 floors	Smart dispatch — nearest car costs 3.6× on average wait and a 230-tick maximum against 108.	Strong on direction · one building
Where the tail matters · hospitals, hotels	Add the fairness weight . Tower max wait 108 → 56; average 18.3 → 22.9. Tune per building.	Strong on tower and burst · weak on offices
Morning up-peak, known lobby	Park idle cars at the lobby. 7.1 → 4.3 average, p90 16 → 11. Should follow time of day.	One pattern · large, mechanistic
Even wear across cars · per-car maintenance	Add the fairness weight — spread falls to 5–18 points on offices, 10 on towers. Do not switch to take-turns: it evens cars by working all of them more.	Direction consistent, nine scenarios · wear not modelled
Saturation · more people than cars can move	No rule fixes a capacity problem. The levers are cars and capacity.	One pattern
Zoning and express cars	Do not zone to cut waits at moderate load: 21.4 → 25.8 average, max 56 → 79. A saturation measure.	One negative result · positive case untested

Limits of the model

NOT MODELLED

Door and acceleration physics — a stop is one tick, whoever boards
Reassignment after dispatch, as hybrid systems allow
Sky-lobby transfers across zones
Passenger no-shows and group entries
Energy

WHAT WOULD SHARPEN THE CONCLUSIONS

A sweep of arrival rate — where each rule breaks down
A sweep of car count — the price of capacity
Fairness weight swept on more than two scenarios
The saturated counterpart to the zoning experiment
Batch re-optimisation in place of greedy per-request choice

None of these would reverse the headline. They would sharpen the secondary findings: where parking, fairness and zoning start and stop paying.

How the numbers were assured

131

tests, six levels

- 1 Unit — every observable assumption
- 2 Structural — no peek-ahead
- 3 Property-based invariants, random buildings
- 4 Golden statistics
- 5 Scenario and CLI, end to end
- 6 JS / Python conformance — incl. per-car usage

40_{/40}

scenario × rule combinations

The browser engine is a JavaScript port. Positions per tick, the full event list and every passenger's lifecycle are asserted identical to Python's.

22

findings, adversarial review

- 0** blockers · 2 major, 20 smaller
- 13** changed code or tests
- 7** changed documents
- 2** acknowledged, no change
- 400** seed fuzz by the reviewer — no invariant broken

How the work was done

SHOT ONE · INTERACTIVE

Owner writes the framing. Assistant asks four questions. Answers become the plan and the working rules.

SHOT TWO · AUTONOMOUS

Assistant executes the plan end to end, committing as it goes; deviations reported in a log. Owner reviews at checkpoints and decides.

ACCEPTED

The eleven ambiguity resolutions; ETD as headline with two baselines; zero-dependency runtime; the seeded generator; explaining the sample case rather than tuning it away.

CHANGED BY THE OWNER

Reframed as a standalone project with the brief as its only input; a time log dropped for one honest statement; sync after every commit; the design notes as a chronological record with outputs.

CORRECTED BY EVIDENCE

Tick order — by unit tests. A LOOK livelock — by a property test. Dwell restarting on every boarding — by the reviewer. Research sentences that outran their sources — traced and revised.

Time: about one hour of my own time framing the problem, writing the plan and reviewing the outputs; then autonomous execution of the plan by the assistant, plus follow-up steering for later changes.

What I would do next

BUILD, IN THIS ORDER

- 1 **Reassignment window** — move a passenger until their car starts slowing, as hybrid systems do.
- 2 **Batch re-optimisation** — revisit outstanding assignments every few ticks.
- 3 **Time-of-day policies** — parking floor and fairness weight follow the traffic profile.

QUESTIONS FOR A REAL CUSTOMER FIRST

What does your traffic look like — arrival rate and mix, by hour?

Which complaint are we fixing: the average wait, or the worst one?

Is a zoned or sky-lobby bank on the table, or one free group?

May the display change a passenger's car after it is shown?

Do people travel in groups, and how often do they not show up?

A rule that accounts for queued stops and the delay it inflicts on others beats rules that do not — increasingly so as buildings get taller and busier.

REPOSITORY

github.com/samuelgregorovic/elevator-dispatch-sim

LIVE SIMULATOR

samuelgregorovic.github.io/elevator-dispatch-sim/docs/viewer/